



GROUP ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT073-3-2-CSLLT

COMPUTER SYSTEM LOW LEVEL TECHNIQUES

APU2F2509CS(AI)

Individual Assignment Report

HAND OUT DATE : 19th March 2026

HAND IN DATE : 15th May 2026

WEIGHTAGE : 60%

Student Name	TP Number
Ng Wei Yang	TP074904

Table of Contents

Table of Contents	i
1 Introduction on Assembly Language	1
2 Research and Analysis: Assembly Language in Cybersecurity	1
2.1 Reverse Engineering and Control Flow	2
2.2 Malware Analysis.....	2
2.3 Buffer Overflow Exploits.....	3
3 System Design and User Manual.....	4
3.1 System Flowchart Diagram.....	4
3.2 User Manual.....	5
3.3 System Assumptions	6
3.4 Known Limitations & Future Works	6
4 System Screenshots.....	7
5 Source Code Explanation.....	14
5.1 Input Sanitization (100-Byte Buffer)	14
5.2 Fixed-Point Decimal Arithmetic	15
5.3 Tiered Memory Architecture (State Management)	16
5.4 Iterative Processing & Control Flow	17
6 Conclusion	18
7 References.....	19

1 Introduction on Assembly Language

Assembly language is a low-level programming language that provides direct correspondence with the computer architecture's machine code instructions. Setting it apart from high-level programming languages where they abstract hardware management, assembly instead controls CPU registers, memory allocation and kernel interrupts. Although nowadays modern applications rarely rely on pure assembly development, understanding its mechanics is important for producing quality code that is highly optimized and resource efficient and key to understanding the underlying architecture of computer systems.

For this assignment, the Netwide Assembler (NASM) for the x86 architecture was used to develop a functional Point of Sale (POS) system for a hypothetical café. The objective was to demonstrate technical competency in using fundamental assembly operations like arithmetic calculations, conditional jumps and loop-based control flows. The system was designed to mimic real-world retail logic, featuring dual-state user roles (Cashier and Manager), dynamic cart math and secure memory management.

2 Research and Analysis: Assembly Language in Cybersecurity

Even though most modern software development are developed using high-level languages, assembly language remains a critical cornerstone in the field of cybersecurity, digital forensics, and malware analysis. This is mainly because malicious software usually bypasses high-level operating system protections and targets the low-level system operations, therefore we require security analysts that utilize reverse engineering to analyse and understand the threats to neutralize them.

2.1 Reverse Engineering and Control Flow

When cybersecurity professionals work with unknown executable files, they usually do not have access to the original source code. Instead, they have to use disassemblers to translate the machine code back into x86 assembly to understand its logic (Yurichev, 2020). A key component of reverse engineering is analysing the control flow of the program, where tracking of the conditional branching instructions like compare (cmp) and jump if equal or not equal (je/jne).

As noted by Andriesse et al. (2016), by tracing these jump instructions, analysts can map out the exact paths a program can take. An example of this is when an analyst traces a specific cmp instruction that checks a user's input against a stored password hash to understand how the core of the software's authentication gate operates. The same logic is reflected in our system to manage access via PIN in our POS system.

2.2 Malware Analysis

Advanced malwares are designed to be stealthy so to avoid detection they avoid creating obvious files on a hard drive by only working within the CPU's short-term memory. This forces analysts to closely monitor how the malware manipulates specific hardware registers such as EAX, EBX and ECX to uncover its payload (Sikorski & Honig, 2012).

By reading these assembly instructions, forensic experts can see what data is being loaded into the accumulator (EAX) before a system interrupt (int 0x80) is called. Leveraging low-level transparency is the only way to determine if a piece of software is attempting to sneakily modify system permissions or steal memory values, much like how the POS system tracks daily revenue exclusively within temporary memory registers before wiping it after final use in Z-Report.

2.3 Buffer Overflow Exploits

A common angle of exploit takes advantage of buffer overflow vulnerabilities, it often revolves around attempting to make a program store more data in its memory buffer than it was designed to hold. Since assembly language provides direct memory access without bounds-checking in high-level languages like Python or Java, an attacker can overflow a buffer to overwrite adjacent memory addresses to inject their own malicious machine code into the executable path (Mohanta & Saldanha, 2020).

To ensure a secure system, this vulnerability was taken account of when designing the POS system, which used a 100-byte input buffer as the defensive measure against this exploit. By explicitly instructing the `sys_read` to capture a maximum of 100 bytes, the software safely catches and discards extra keystrokes and protects the choice memory block from being overflowed and causing unwanted behaviour in the system.

3 System Design and User Manual

3.1 System Flowchart Diagram

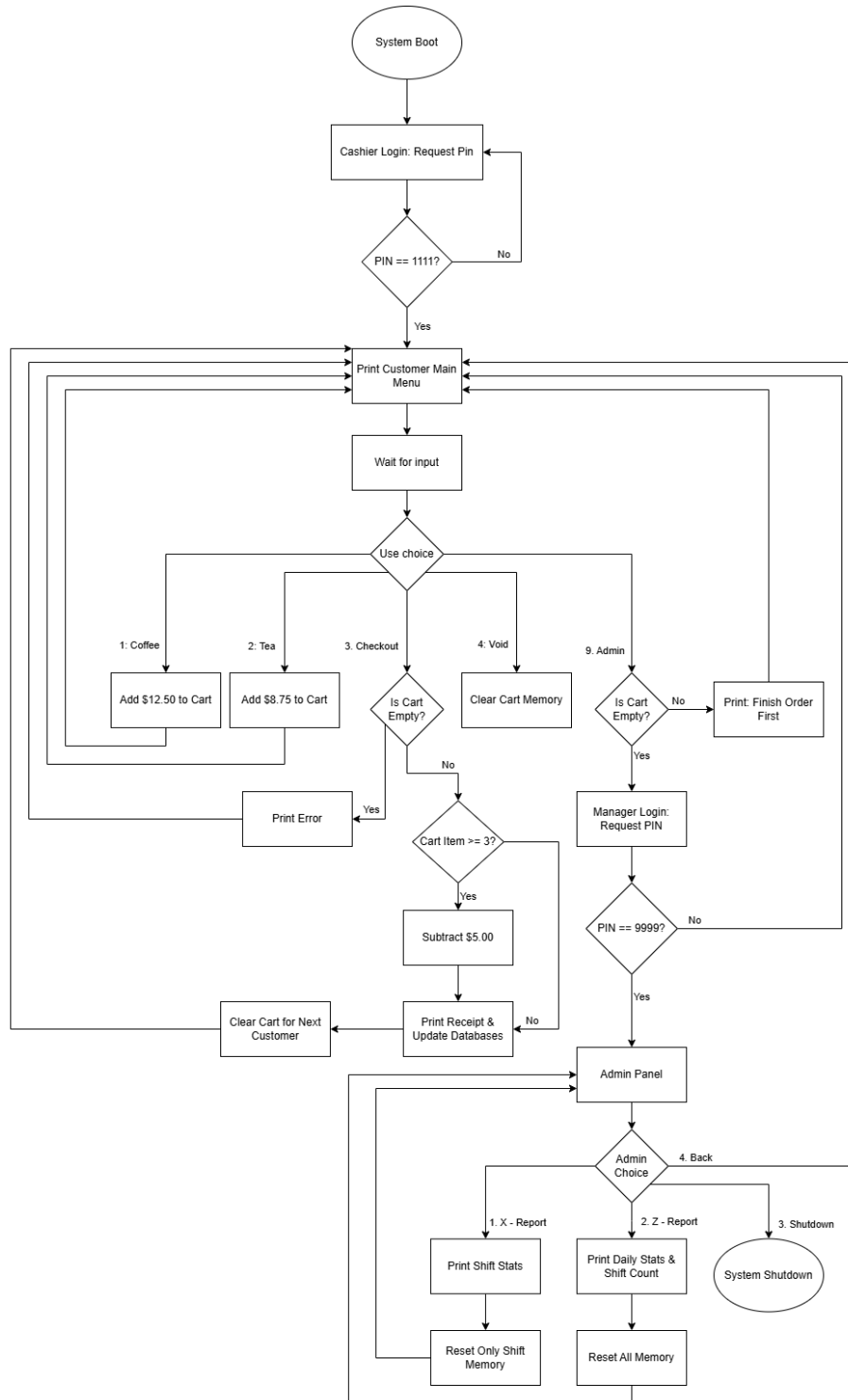


Figure 1: System Flowchart

The figure above shows the flowchart of my Café Point of Sale (POS) system which was designed to handle two states, separating standard customer checkout operations and the managerial reporting features. The system logic ensures separation of authorization to the system by employing PIN and conditional jumps.

Furthermore, to ensure the math and numbers are tracked accurately, the system uses different sets of clearly named variables like current cart, the shift totals, daily totals, etc. Moreover, the system rejects if a cashier tries to access the administrative state while the cart is not empty, this was implemented to ensure data integrity that might've occurred from differing shifts while also keeping the business logic sound. This system was inspired by my meagre interactions while as doing quality assurance testing in my internship, namely the Velos and Slurp POS systems.

3.2 User Manual

The system operates using numerical terminal inputs to navigate and access functionalities. It is also divided into two distinct user roles, namely Cashier and Manager.

Cashier Operations:

1. **System Boot:** Upon launching the system, it is locked and the cashier must enter the 4-digit Cashier PIN (1111) to access the main terminal.
2. **Taking Orders:** The cashier enters 1 to add a Premium Coffee (\$12.50) or 2 to add a Matcha Latte (\$8.75), which updates the short-term cart memory.
3. **Voiding Orders:** If a mistake was made, entering 4 will instantly void the current order and clears out the short-term cart memory without affecting the other variables.
4. **Checkout:** After the order is taken, the cashier enters 3 to process the payment. If the cart contains 3 or more total items, the system automatically applies a \$5 bulk discount. Then a receipt is printed and the totals are pushed to the shift and daily databases for managerial purposes. Which then clears the cart memory for the next customer's order. An error message was implemented if the Cashier tried to access checkout while the cart is empty to avoid confusion of an unresponsive system.

Manager Operations:

1. **Admin Access:** From the main menu, entering 9 will request access to Manager's side of the system which ensures that the cart is empty first and the proper PIN (9999) has been corrected inputted.

2. **X-Report (Shift Change):** Enters 1 to generate an X-Report which prints the accumulated sales for the current shift and increments the daily shift counter and resets the Shift Memory for the next cashier.
3. **Z-Report (End of Day):** Enter 2 to generate a Z-Report which prints the total accumulated sales across all shifts for the entire day. Then it safely executes a complete system memory wipe, zeroing out all databases and resetting the physical drawer float to prepare the system for the next business day.
4. **Shutdown:** Entering 3 safely terminates the program.

3.3 System Assumptions

A few logical assumptions were made during the system design phase to ensure a streamlined and stable system.

1. **Manual Change Handling:** It is assumed that while the system can accurately track total revenue and expected physical drawer with an opening balance of \$100.00 float and the cashier will be responsible for manually calculating and dispensing the correct cash change to the customer and will be held responsible for any differences or missing money.
2. **Input Security Environment:** It is assumed that the POS terminal operates in a physically retail environment while supervised and handled by trained cashiers. As a result, the only measure implemented for input sanitization was a 100-byte read buffer to swallow accidental multi-key inputs while preventing malicious buffer overflow attacks.
3. **Shift Changes:** It is assumed that the manager will take note of the X-Report when shift change occurs and verifies with the cashier to double confirm that the notes taken are accurate then dismisses them, ensuring fair accountability.

3.4 Known Limitations & Future Works

The current system only allows for the manager to safely shut down the terminal at any time using option 3 in the admin menu. While this is a necessary feature for emergency reboots, for future improvements, an implementation that checks if the daily sales memory registers are empty and warns the manager if they still want to continue to shut down the terminal before executing a Z-Report would be ideal. Mainly to prevent accidental data loss at closing time. Moreover, the current system lacks the rate limiting of PIN attempts. For future enhancements

a incrementing loop would be implemented to trigger a timeout system after consecutive failed attempts to prevent brute-force attacks.

4 System Screenshots

```
Terminal locked. PIN: 1110
Access denied.

Terminal locked. PIN: 1111

--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): █
```

Figure 2: System Boot & Authentication

The figure above shows entering of a wrong PIN and an unauthorized access message, followed by the entering of the right PIN (1111), the user successfully passes the cmp verification and accessed the main menu loop.

```
--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): 1
Added Premium Coffee.

--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): 2
Added Matcha Latte.
```

Figure 3: Adding to Cart & Arithmetic

The figure above shows the adding of premium coffee and matcha latte to the order. The system dynamically updates the short-term memory registers using add and inc arithmetic instructions as items are chosen.

```
--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): 3

Discount applied (-$5.00)

--- Receipt ---
Coffee: 2
Tea   : 1
Total : $28.75

--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): █
```

Figure 4: Checkout & Discount Feature

The figure above shows the checkout sequence where a \$5.00 discount was applied since there are at least 3 items in the cart. The system utilizes a `jl` condition to check the item quantity and apply the discount before formatting the fixed-point integers into a readable decimal currency output.

```
--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): 1
Added Premium Coffee.

--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): 9
Finish or void current order first.

--- Cafe POS ---
1: Premium Coffee ($12.50)
2: Matcha Latte ($8.75)
3: Checkout
4: Void Order
Choice (9 for Admin): █
```

Figure 5: Transaction Lock

The figure above shows the user trying to access the admin menu while the cart was not empty, demonstrating the transaction lock preventing access to the manager state while the short-term cart memory is actively holding uncleared data.

```
--- Cafe POS ---  
1: Premium Coffee ($12.50)  
2: Matcha Latte ($8.75)  
3: Checkout  
4: Void Order  
Choice (9 for Admin): 4  
Order voided.  
  
--- Cafe POS ---  
1: Premium Coffee ($12.50)  
2: Matcha Latte ($8.75)  
3: Checkout  
4: Void Order  
Choice (9 for Admin): 3  
Cart is empty. Add items first.
```

Figure 6: Void & Responsiveness

The figure above shows the void order function which clears the cart memory if any mistakes were made while the cashier was taking the order. It also shows a reminder message when cashier tries to use the checkout function while the cart is empty to show the system is operational and responsive and it was a user error.

```
--- Cafe POS ---  
1: Premium Coffee ($12.50)  
2: Matcha Latte ($8.75)  
3: Checkout  
4: Void Order  
Choice (9 for Admin): 9  
  
Manager PIN: 1239  
Access denied.  
  
--- Cafe POS ---  
1: Premium Coffee ($12.50)  
2: Matcha Latte ($8.75)  
3: Checkout  
4: Void Order  
Choice (9 for Admin): 9  
  
Manager PIN: 9999  
  
--- Admin ---  
1: X-Report (End Shift)  
2: Z-Report (End Day)  
3: Shutdown  
4: Back  
Choice: █
```

Figure 7: Admin Access & Menu

The figure above shows a failed attempt to access the admin menu and the access denied message, returning the user back to the main menu. Then a successful attempt which shows the admin menu.

```
--- Admin ---  
1: X-Report (End Shift)  
2: Z-Report (End Day)  
3: Shutdown  
4: Back  
Choice: 1  
  
--- X-REPORT (SHIFT SUMMARY) ---  
Coffee: 2  
Tea   : 1  
Total : $28.75  
Shift closed. Memory reset for next cashier.
```

Figure 8: Shift Summary

The figure above shows the X-Report function to generate shift summary about the clocking out cashier's summarized orders during their shift. Allowing the manager to take note of the amount accrued during their shift.

```
--- Admin ---  
1: X-Report (End Shift)  
2: Z-Report (End Day)  
3: Shutdown  
4: Back  
Choice: 2  
  
--- Z-REPORT (DAILY SUMMARY) ---  
  
Total Shifts Today: 2  
Coffee: 2  
Tea   : 1  
Total : $28.75  
Drawer Cash: $128.75  
Cash removed. Drawer reset to $100.00.
```

Figure 9: Daily Summary

The figure above shows the Z-Report function to generate the daily summary about the whole day's transactions including number of total shifts so the manager can verify with their notes ensuring no one skipped a shift summary. It also includes the total expected cash in the physical cash drawer so the manager can double check for any missing money and trace it back to the responsible cashier using the notes from X-Report.

```
--- Admin ---  
1: X-Report (End Shift)  
2: Z-Report (End Day)  
3: Shutdown  
4: Back  
Choice: 4  
  
--- Cafe POS ---  
1: Premium Coffee ($12.50)  
2: Matcha Latte ($8.75)  
3: Checkout  
4: Void Order  
Choice (9 for Admin):
```

Figure 10: Back Function

The figure above shows the back function from the admin menu, essentially logging out of the admin state and giving the next cashier their shift operations swiftly.

```
--- Admin ---  
1: X-Report (End Shift)  
2: Z-Report (End Day)  
3: Shutdown  
4: Back  
Choice: 3  
Shutting down...
```

Figure 11: System Shutdown

The figure above shows the system shutting down and ending the program.

5 Source Code Explanation

5.1 Input Sanitization (100-Byte Buffer)

```
mov eax, 3
mov ebx, 0
mov ecx, choice
mov edx, 100
int 0x80

; check manager pin 9999
mov al, [choice]
cmp al, '9'
jne admin_denied
mov al, [choice+1]
cmp al, '9'
jne admin_denied
mov al, [choice+2]
cmp al, '9'
jne admin_denied
mov al, [choice+3]
cmp al, '9'
jne admin_denied
jmp admin_loop
```

Figure 12: Input Buffer & Authorization Validation

The figure above shows the code block that takes an input with a buffer of 100-byte (mov edx, 100) and validation block to check the inputted PIN for authorized access to the admin menu. The 100-byte input buffer was implemented to avoid critical failure caused by multi-character inputs or the [Enter] key, it captures and cmp only the first 4 characters and discards all extra inputs on the input stream. This security measure prevents buffer overflow vulnerabilities where malicious actors might exploit and attempt to input a massive string to overwrite adjacent memory addresses and force unauthorized system behaviour.

5.2 Fixed-Point Decimal Arithmetic

```
; divide to print decimals
print_currency:
    pusha
    mov edx, 0
    mov ebx, 100
    div ebx

    push edx
    call print_number

    ; add the dot
    mov eax, 4
    mov ebx, 1
    mov ecx, decimal_pt
    mov edx, 1
    int 0x80

    ; format single digit cents
    pop eax
    cmp eax, 10
    jge .print_cents

    push eax
    mov eax, 4
    mov ebx, 1
    mov ecx, zero_char
    mov edx, 1
    int 0x80
    pop eax
```

Figure 13: Decimal Handling

The figure above shows how the system handles decimal points in the currency. To avoid precision errors and extreme complexity floating point units in x86 assembly, every item is tracked as whole integers in cents. Only when displaying the amount the `print_currency` subroutine is called and handles conversion by dividing by 100 and checking for single digit cents and adding padding when necessary and finally printing the output to the terminal screen.

5.3 Tiered Memory Architecture (State Management)

```
mov dword [total], 0
mov dword [coffee_qty], 0
mov dword [tea_qty], 0
jmp menu_loop
```

Figure 14: Cart Memory Wipe

```
mov dword [shift_total], 0
mov dword [shift_coffee], 0
mov dword [shift_tea], 0
jmp admin_loop
```

Figure 15: Shift Memory Wipe

```
mov dword [shift_total], 0
mov dword [shift_coffee], 0
mov dword [shift_tea], 0
mov dword [daily_total], 0
mov dword [daily_coffee], 0
mov dword [daily_tea], 0
mov dword [shift_count], 0
mov dword [drawer_balance], 10000
jmp admin_loop
```

Figure 16: System Data Wipe

The three figures above show that data integrity is maintained by isolating variables in the .bss section into distinct tiers, short-term for active customer cart, mid-term for totals of current shift and long-term for daily totals. After checking out the short-term cart memory is added to the shift memory (mid-term memory) and daily memory (long-term memory) then its wiped cleaned for the next customer's order. Then after a shift change occurs, the shift memory resets for the next cashier. Finally, after Z-Report prints the daily summary, all memory variables are forcefully zeroed out and overwritten using the `mov dword` instruction, safely preparing everything for a brand-new business day.

5.4 Iterative Processing & Control Flow

```
menu_loop:
    mov eax, 4
    mov ebx, 1
    mov ecx, menu
    mov edx, menu_len
    int 0x80

    mov eax, 3
    mov ebx, 0
    mov ecx, choice
    mov edx, 100
    int 0x80

    ; see what user picked
    mov al, [choice]
    cmp al, '1'
    je add_coffee
    cmp al, '2'
    je add_tea
    cmp al, '3'
    je checkout
    cmp al, '4'
    je void_order
    cmp al, '9'
    je try_admin_login
    jmp menu_loop
```

Figure 17: Main Menu Loop

The figure above shows the main menu loop which uses conditional (je) to execute the chosen procedures like adding an item, checking out and voiding orders and navigating to the admin menu. With the fallback of the unconditional jump at the end of the procedure, if an unexpected input was received, it will loop back to the start and ensures the program runs smoothly until a shutdown command is explicitly issued.

6 Conclusion

While developing this Point of Sale (POS) system in x86 Assembly, I gained better understanding of the foundational mechanism of computer architecture. I had to shift logic from high-level languages to explicit hardware management, making sure all memory registers are allocated, tracked and cleared accordingly. Through system interrupts and raw CPU instructions, the implementation of fixed-point arithmetic loops and buffer sanitization were successful in achieving a functional and secure system. Ultimately, understanding how the processor handles memory boundaries and instruction execution at the lowest level help build a solid foundation for me to write cleaner and more optimized software in my future, especially when building complex systems.

7 References

Andriesse, D., Chen, X., van der Veen, V., Slowinska, A., & Bos, H. (2016). An in-depth analysis of disassembly on full-scale x86/x64 binaries. Proceedings of the 25th USENIX Security Symposium, 583–600.

<https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/andriesse>

Mohanta, A., & Saldanha, A. (2020). Malware analysis and detection engineering: A comprehensive approach to detect and analyze modern malware. Apress.

<https://doi.org/10.1007/978-1-4842-6193-4>

Sikorski, M., & Honig, A. (2012). Practical malware analysis: The hands-on guide to dissecting malicious software. No Starch Press. <https://nostarch.com/malware>

Yurichev, D. (2020). Reverse engineering for beginners. <https://beginners.re/>